

Flask JWT Authentication

Complete Code Explanation & Workflow

Based on the Flask application discussed in the conversation

1. What This Application Does

This Flask application is a simple REST API demonstrating JWT (JSON Web Token) authentication. A user logs in with a username and password. If the credentials are valid, the server creates a JWT access token. The client then sends that token with requests to protected endpoints such as `/profile` and `/dashboard`.

```
User
↓
POST /login with username + password
↓
Flask verifies credentials
↓
JWT access token is created
↓
Client receives JWT
↓
Client sends JWT with protected requests
↓
Flask validates JWT
↓
Private API data is returned
```

2. Endpoints in the Application

Endpoint	Method	JWT Required?	Purpose
/	GET	No	Check that the Flask API is running
/login	POST	No	Authenticate user and generate JWT
/profile	GET	Yes	Access private profile data
/dashboard	GET	Yes	Access protected dashboard data

3. Imports and Their Purpose

```
from flask import Flask, request, jsonify

from flask_jwt_extended import (
    JWTManager,
    create_access_token,
    jwt_required,
    get_jwt_identity
)

from datetime import timedelta

from dotenv import load_dotenv
import os
```

- Flask: Creates the web application and API routes.
- request: Reads data sent by the client, such as the JSON login body.
- jsonify: Returns JSON responses.
- JWTManager: Connects Flask-JWT-Extended to the Flask application.
- create_access_token: Creates a JWT after successful authentication.
- jwt_required: Protects an endpoint so a valid JWT is required.
- get_jwt_identity: Reads the identity stored inside the JWT.
- timedelta: Used to define token expiration time.
- load_dotenv and os: Load the JWT secret key from the .env file.

4. Loading the Secret Key

```
load_dotenv()

mykey = os.getenv("JWT_SECRET_KEY")
```

load_dotenv() reads values from a .env file. The application then reads JWT_SECRET_KEY from the environment.

```
JWT_SECRET_KEY=my-super-secret-jwt-key-12345
```

For a real application, keep the secret key outside the source code and use a strong secret value. The fallback key in the demo is useful for learning but should not be used for production.

5. Creating and Configuring the Flask Application

```
app = Flask(__name__)

app.config["JWT_SECRET_KEY"] = mykey or "this-is-my-key-test-code-fordemo"
app.config["JWT_ACCESS_TOKEN_EXPIRES"] = timedelta(minutes=15)

jwt = JWTManager(app)
```

- Flask(__name__) creates the Flask application.
- JWT_SECRET_KEY is used to sign and verify JWTs.
- JWT_ACCESS_TOKEN_EXPIRES sets the access-token lifetime to 15 minutes.
- JWTManager(app) activates JWT functionality for this Flask application.

6. Dummy Users

```
users = {
    "karthik": {
        "id": 1,
        "username": "karthik",
        "password": "1234"
    },
    "admin": {
        "id": 2,
        "username": "admin",
        "password": "admin123"
    }
}
```

For this training example, users are stored in a Python dictionary. In a real application, users would normally be stored in a database such as Oracle, MySQL, PostgreSQL, SQL Server, etc.

Also note: passwords should not be stored as plain text in a production system. Password hashes should be used instead.

7. Root Endpoint: GET /

```
@app.route("/")
def home():
    return jsonify({
        "message": "Flask JWT API is running"
    })
```

This is a public endpoint because it does not have `@jwt_required()`. It can be opened directly in a browser.

`http://127.0.0.1:5000/`

Expected response:

```
{
  "message": "Flask JWT API is running"
}
```

8. Login Endpoint: POST /login

```
@app.route("/login", methods=["POST"])
def login():
```

The login endpoint accepts POST requests only. This is why opening `/login` directly in a browser produces '405 Method Not Allowed': the browser normally sends a GET request.

POST <http://127.0.0.1:5000/login>

9. Reading the Login JSON Body

```
data = request.get_json(silent=True)

if not data:
    return jsonify({
        "message": "Request body must contain JSON",
        "example": {
            "username": "karthik",
            "password": "1234"
        }
    }), 400
```

The client sends JSON containing the username and password. If the request body does not contain usable JSON, the application returns HTTP 400.

10. Extracting Username and Password

```
username = data.get("username")
password = data.get("password")

{
    "username": "karthik",
    "password": "1234"
}
```

The username and password are extracted from the JSON request.

11. Checking the Username

```
if username not in users:
    return jsonify({
        "message": "Invalid username or password"
    }), 401
```

If the username is not present in the users dictionary, the server returns HTTP 401 Unauthorized.

12. Checking the Password

```
user = users[username]

if user["password"] != password:
    return jsonify({
        "message": "Invalid username or password"
    }), 401
```

If the password does not match the stored password, authentication fails with HTTP 401.

13. Creating the JWT

```
access_token = create_access_token(
    identity=str(user["id"])
)
```

After successful authentication, the server creates a JWT. In this application, the user's ID is stored as the JWT identity. For Karthik, the identity is '1'.

```
Username + Password
    ↓
Credential validation
    ↓
User ID = 1
    ↓
create_access_token()
    ↓
Signed JWT token
```

14. Returning the JWT to the Client

```
return jsonify({
    "message": "Login successful",
    "access_token": access_token
}), 200
```

The client receives the access token. In Postman, copy the access_token value from the response.

```
{
    "message": "Login successful",
    "access_token": "eyJhbGciOiJIUzI1NiIs..."
}
```

15. Protected Profile Endpoint: GET /profile

```
@app.route("/profile", methods=["GET"])
@jwt_required()
def profile():

    user_id = get_jwt_identity()

    return jsonify({
        "message": "You are authorized",
        "user_id": user_id,
        "profile": "This is private profile data"
    }), 200
```

@jwt_required() is the key security decorator. It tells Flask-JWT-Extended to validate the JWT before profile() is executed.

If no Authorization header is sent, the request is rejected. If the token is invalid or expired, the request is rejected. If the token is valid, profile() executes.

16. How to Call /profile

Use Postman for this protected endpoint. The browser URL alone does not automatically send your JWT Authorization header.

```
GET http://127.0.0.1:5000/profile
```

```
Authorization:
```

```
Bearer <your JWT token>
```

In Postman, select Authorization → Bearer Token and paste only the JWT token. Postman creates the Authorization: Bearer ... header.

17. get_jwt_identity()

```
user_id = get_jwt_identity()
```

During login, the application stored the user's ID as the JWT identity. `get_jwt_identity()` retrieves that value after the token has been successfully validated.

```
Login:
```

```
identity = "1"
```

```
Later:
```

```
get_jwt_identity() → "1"
```

18. Protected Dashboard Endpoint

```
@app.route("/dashboard", methods=["GET"])
@jwt_required()
def dashboard():

    user_id = get_jwt_identity()

    return jsonify({
        "message": "Welcome to dashboard",
        "user_id": user_id,
        "data": [
            "Account details",
            "reports"
        ]
    }), 200
```

The dashboard follows the same JWT authentication process as `/profile`. A valid JWT is required.

19. Starting the Flask Server

```
if __name__ == "__main__":  
    app.run(debug=True)
```

When `app.py` is executed directly with `python app.py`, Flask starts the development server. The default address is `http://127.0.0.1:5000`.

```
python app.py
```

20. Complete End-to-End Workflow

STEP 1

Start Flask

↓

```
python app.py
```

↓

`http://127.0.0.1:5000`

STEP 2

Browser or Postman

↓

GET /

↓

Flask API is running

STEP 3

Postman

↓

POST /login

↓

username + password

↓

Validate credentials

↓

Create JWT

↓

Return access_token

STEP 4

Postman

↓

GET /profile

+

Authorization: Bearer <JWT>

↓

@jwt_required()

↓

Validate token

↓

get_jwt_identity()

↓

Return private profile data

STEP 5
Postman
↓
GET /dashboard
+
Authorization: Bearer <JWT>
↓
JWT validation
↓
Return protected dashboard data

21. Public vs Protected APIs

Type	Endpoint	Authentication
Public	/	Not required
Authentication	/login	Username/password
Protected	/profile	JWT required
Protected	/dashboard	JWT required

22. Why /profile Cannot Simply Be Opened in the Browser

Typing `http://127.0.0.1:5000/profile` in a normal browser sends a GET request but normally does not include your JWT Authorization header. Because `/profile` has `@jwt_required()`, Flask returns an authentication error such as 'Missing Authorization Header'.

Browser:
GET /profile
(no JWT)
↓
`@jwt_required()`
↓
Rejected

Postman:
GET /profile
Authorization: Bearer <JWT>
↓
`@jwt_required()`
↓
JWT validated
↓
Profile returned

23. Recommended Postman Testing Sequence

- 1. Start Flask with python app.py.
- 2. Test GET http://127.0.0.1:5000/ in the browser or Postman.
- 3. In Desktop Postman, send POST http://127.0.0.1:5000/login.
- 4. Use Body → raw → JSON with username 'karthik' and password '1234'.
- 5. Copy the access_token from the login response.
- 6. Create a new GET request for http://127.0.0.1:5000/profile.
- 7. Select Authorization → Bearer Token and paste the JWT.
- 8. Click Send and verify the authorized profile response.
- 9. Repeat the same process for GET /dashboard.

24. Common Errors and Their Meaning

Error	Likely Cause	Fix
405 Method Not Allowed	Wrong HTTP method	Use POST for /login and GET for /profile.
Missing Authorization Header	No JWT sent	Add Authorization → Bearer Token in Postman.
Invalid username or password	Credentials do not match	Use a valid user from the demo dictionary.
Token expired	JWT lifetime has passed	Login again to obtain a new token.
Invalid token	JWT is malformed/changed/incorrect	Copy the complete token from /login.

25. Complete Code

```
from flask import Flask, request, jsonify

from flask_jwt_extended import (
    JWTManager,
    create_access_token,
    jwt_required,
    get_jwt_identity
)

from datetime import timedelta

from dotenv import load_dotenv
import os

load_dotenv()

# Read secret key from .env
mykey = os.getenv("JWT_SECRET_KEY")

# Initialize Flask
app = Flask(__name__)

# Configure JWT
app.config["JWT_SECRET_KEY"] = mykey or "this-is-my-key-test-code-fordemo"
app.config["JWT_ACCESS_TOKEN_EXPIRES"] = timedelta(minutes=15)

# Initialize JWT
jwt = JWTManager(app)

# Dummy users
users = {
    "karthik": {
        "id": 1,
        "username": "karthik",
        "password": "1234"
    },
    "admin": {
        "id": 2,
        "username": "admin",
        "password": "admin123"
    }
}

# Home
@app.route("/")
def home():
    return jsonify({
        "message": "Flask JWT API is running"
    })

# Login
@app.route("/login", methods=["POST"])
def login():

    data = request.get_json(silent=True)
```

```

if not data:
    return jsonify({
        "message": "Request body must contain JSON",
        "example": {
            "username": "karthik",
            "password": "1234"
        }
    }), 400

username = data.get("username")
password = data.get("password")

if username not in users:
    return jsonify({
        "message": "Invalid username or password"
    }), 401

user = users[username]

if user["password"] != password:
    return jsonify({
        "message": "Invalid username or password"
    }), 401

access_token = create_access_token(
    identity=str(user["id"])
)

return jsonify({
    "message": "Login successful",
    "access_token": access_token
}), 200

# Protected profile
@app.route("/profile", methods=["GET"])
@jwt_required()
def profile():

    user_id = get_jwt_identity()

    return jsonify({
        "message": "You are authorized",
        "user_id": user_id,
        "profile": "This is private profile data"
    }), 200

```

```

# Protected dashboard
@app.route("/dashboard", methods=["GET"])
@jwt_required()
def dashboard():

    user_id = get_jwt_identity()

    return jsonify({
        "message": "Welcome to dashboard",
        "user_id": user_id,
        "data": [
            "Account details",
            "reports"
        ]
    }), 200

if __name__ == "__main__":
    app.run(debug=True)

```

26. Key JWT Concepts to Remember

- Authentication: `/login` verifies who the user is.
- JWT generation: `create_access_token()` creates the signed access token.
- Authorization: `@jwt_required()` protects private endpoints.
- Identity: `get_jwt_identity()` retrieves the user identity from the JWT.
- Bearer token: The client sends the JWT in the Authorization header.
- Expiration: This application configures access tokens to expire after 15 minutes.
- Stateless access: The client sends the token with protected requests instead of sending the password again.